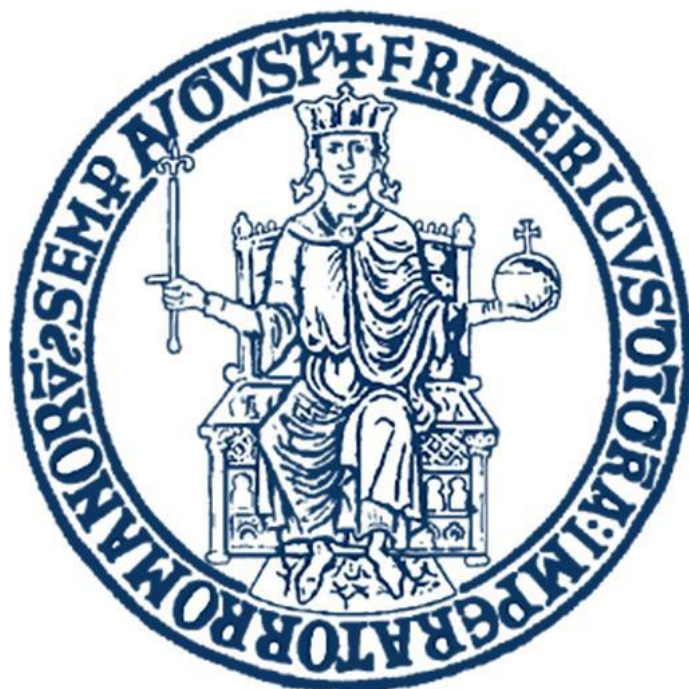




UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DOCUMENTAZIONE DI DESIGN DEL SOFTWARE
PER PROGETTO DI INGEGNERIA DEL SOFTWARE

CdL Triennale in Informatica

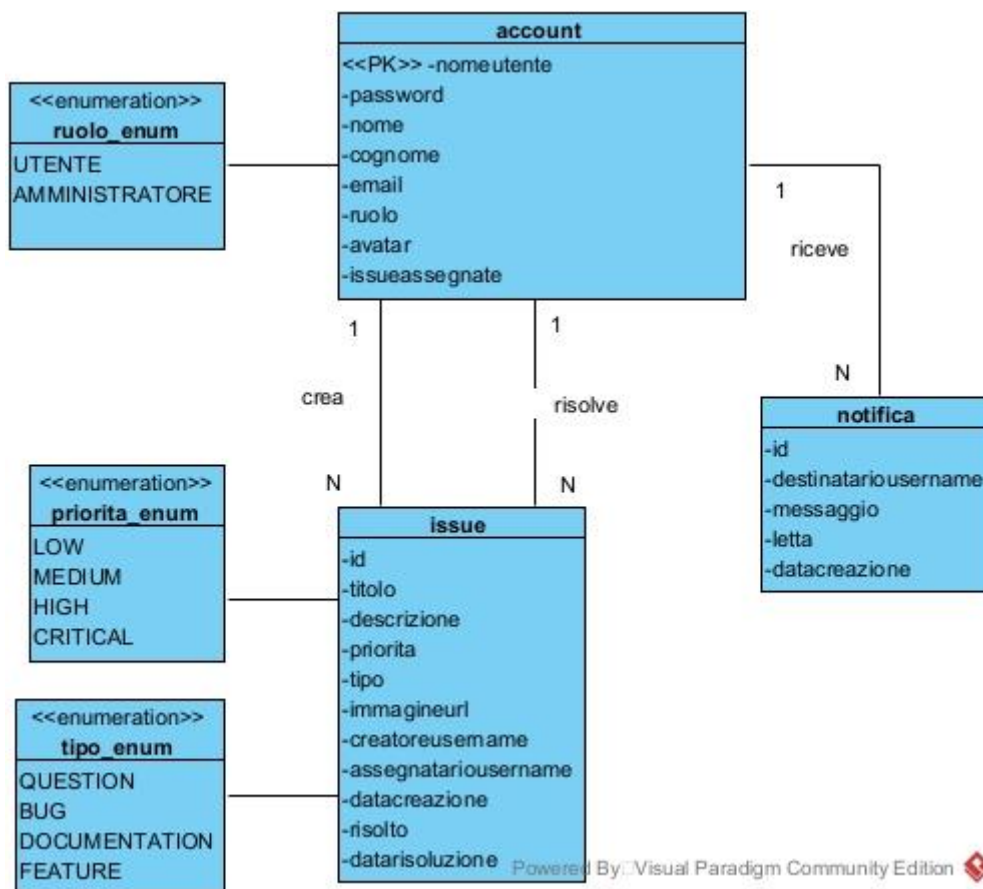
AGOSTINO SORRENTINO
N86005123
MARIATERESA PRINCIPATO
N86005284

ANNO ACCADEMICO: 2025/2026

SOMMARIO

2	Descrizione dello schema per la persistenza dei dati	3
3	Diagramma delle classi di design	4
4	Descrizione e motivazione delle scelte di software di design adottate.....	5
5	Uso di strumenti di versioning.....	6
5.1	Report di qualità del codice generati da SonarQube (solo per il backend)	6

1 DESCRIZIONE DELLO SCHEMA PER LA PERSISTENZA DEI DATI



Lo schema del database è stato progettato seguendo i principi della normalizzazione per garantire l'integrità dei dati. Abbiamo tuttavia mantenuto delle denormalizzazioni allo scopo di migliorare le performance di lettura e semplicità delle funzioni.

Per i campi ruolo, priorit  e tipo abbiamo utilizzati i tipi ENUM nativi di PostgreSQL. Ciò semplifica le query e garantisce l'integrità dei dati a livello del database bloccando l'inserimento di valori non validi senza usare chiavi esterne.

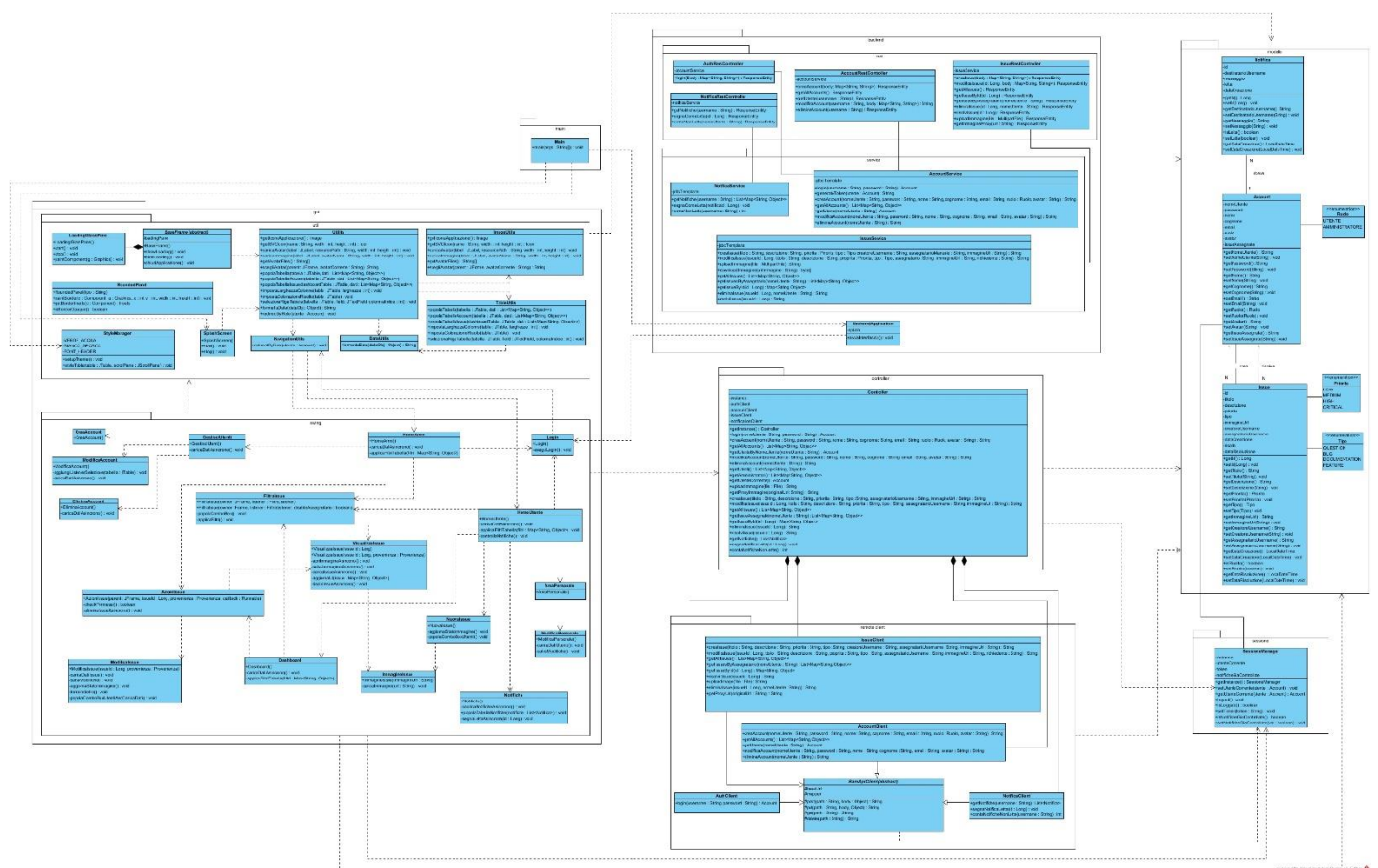
Nella tabella account è presente il campo issueAssegnate che tiene traccia del valore di "peso" totale di issue attive per ogni utente. Il peso è calcolato a seconda di quante issue siano assegnate all'utente e il valore di criticità: LOW=1, MEDIUM=2, HIGH=3. CRITICAL=4. Questo valore è utilizzato dall'algoritmo di assegnazione automatica (trova_utente_libero). Calcolarlo ogni volta con una COUNT sarebbe costoso: mantenendolo aggiornato con un Trigger la lettura diventa istantanea.

Abbiamo lasciato al database la responsabilità di mantenere la coerenza dei dati derivati. Ciò fa in modo che tutte le regole critiche siano centralizzate: anche se un bug nel backend Java tentasse di bypassare la logica, il database garantirebbe la correttezza. Inoltre riduce il traffico di rete.

Le immagini sono salvate nel database solo e unicamente come URL che puntano a Supabase Storage. Salvare solo il riferimento mantiene il database veloce e leggero.

Tutte le relazioni sono protette da vincoli di chiave esterna. Inoltre, è stata applicata la politica ON DELETE CASCADE per le notifiche (se si elimina un utente, le sue notifiche sono inutili) e controlli bloccanti (simulato via trigger) per gli account con issue attive per evitare di lasciare issue orfane.

2 DIAGRAMMA DELLE CLASSI DI DESIGN



3 DESCRIZIONE E MOTIVAZIONE DELLE SCELTE DI SOFTWARE DI DESIGN ADOTTATE

Il design è stato guidato dalla richiesta di creare un sistema modulare, manutenibile e testabile. Abbiamo adottato un approccio dedito alla separazione delle responsabilità.

Il sistema è diviso in layer orizzontali che comunicano solo con il livello adiacente. Quest struttura permette di modificare un livello senza impattare gli altri: potremmo cambiare l'interfaccia grafica senza toccare una riga del backend, o cambiare il database senza modificare il client.

Abbiamo implementato due singleton (Controller e SessioneManager). Il Controller garantisce che ci sia un unico punto di accesso per tutte le operazioni verso il backend, centralizzando la gestione dei client HTTP. SessioneManager assicura che lo stato dell'utente loggato sia unico e coerente in tutta l'applicazione: evita di dover passare l'oggetto "Utente" come parametro a ogni singola finestra.

Abbiamo anche semplificato le interfacce tramite le classi Controller e Utility. Il controller nasconde la complessità dei vari client (AuthClient, AccountClient, IssueClient, NotificationClient): per esempio, la GUI chiama solo "controller.creaIssue()" senza sapere che c'è una chiamata HTTP, una serializzazione JSON e una gestione degli errori. La classe Utility invece raggruppa metodi statici di diverse classi helper (ImageUtils, TableUtils, DateUtils, NavigationUtils) offrendo un'unica interfaccia semplice per le operazioni comuni della UI. Questo riduce l'accoppiamento tra la GUI e la logica di rete/utility. Rende il codice delle finestre più pulito e leggibile.

Per il riutilizzo del codice abbiamo le classi "BaseApiClient" e "BaseFrame". BaseApiClient definisce lo "scheletro" delle chiamate HTTP lasciando alle sottoclassi solo il compito di definire il path e il body. BaseFrame definisce il comportamento standard di tutte le finestre (icona, chiusura, glasspane per il caricamento) che viene ereditato da tutte le schermate (tranne i popup JDialog). Ciò elimina la duplicazione del codice e rende la modifica di queste feature più facile da effettuare, dato che va modificato solo una volta per tutte le classi.

Abbiamo implementato anche la gestione di eventi in asincrono: le operazioni più lunghe (di rete) vengono eseguite in un thread separato (doInBackground) e, al termine, viene notificato il thread della UI tramite "done". Questo, oltre a permettere di dare feedback all'utente quando effettua un'operazione senza che la schermata si blocchi, rende i componenti testabili. Nei test abbiamo

potuto iniettare un mock del JdbcTemplate nel Service proprio grazie a questo pattern, isolando la logica dal database vero e proprio.

Le classi nel modello, infine, sono puri contenitori di dati usati per il trasporto tra client e server. Disaccoppiano la struttura interna del database dalla logica applicativa. L'uso di oggetti Java, oltre a rispettare la consegna di programmazione orientata ad oggetti (Object Oriented), assicura la tipizzazione forte rispetto all'uso di Map o ResultSet.

4 USO DI STRUMENTI DI VERSIONING

Per la realizzazione e per il versioning di questo progetto è stato utilizzato GitHub. Il codice, la documentazione, Javadoc e il file jar eseguibile è disponibile all'indirizzo <https://github.com/aaagoo/BugBoard26>.

4.1 REPORT DI QUALITÀ DEL CODICE GENERATI DA SONARQUBE (SOLO PER IL BACKEND)

